

# zØmbie zØne

*Parc d'attractions fictif post-apocalyptique*

---

Titre Professionnel

**CDA — Concepteur Développeur d'Applications — Niv. 6**

Candidat

**Stéphane Rochard**

Formation

**O'clock — RNCP37873**

Soutenance

**9 juin 2026**

Production

**sharo.fr | [sharo.fr/api/docs](https://sharo.fr/api/docs)**

# Plan de la présentation

---

01

Projet & candidat

02

Cahier des charges — Visiteur / Membre

03

Cahier des charges — Admin & contraintes

04

Gestion de projet SCRUM

05

Architecture logicielle

06

Modèle de données (MCD)

07

Stack technique justifiée

08

Réalisations — Authentification JWT

09

Réalisations — Code & sécurité

10

Plan de tests — 151 tests

11

Jeu d'essai — Cycle JWT

12

Déploiement — CI/CD & pipeline

13

Déploiement — Docker & VPS

14

Difficultés techniques résolues

15

Améliorations futures

16

Compétences AT1 — Développer

17

Compétences AT2 & AT3

18

Conclusion

# 01 — Projet & Candidat

## Le projet zØmbie zØne

- Site de réservation d'un parc fictif post-apocalyptique
- Vitrine immersive + espace client + backoffice admin
- Réalisé en solo — contexte professionnel simulé
- Déployé en production : sharo.fr
- 4 acteurs : Visiteur, Membre, Admin, Système
- Monorepo TypeScript — backend + frontend

## Rôles endossés (solo)

### Product Owner

*Backlog, user stories, priorisation*

### Scrum Master

*Sprints hebdo, GitHub Projects*

### Architecte

*Stack, patterns, conventions*

### Dev Back-End

*API REST, JWT, BDD, sécurité*

### Dev Front-End

*SPA React, Zustand, routing*

### DevOps / QA

*Tests, Docker, CI/CD, VPS*

## 02 — Cahier des charges — Visiteur & Membre

Cibles : Ados / jeunes adultes 16-30 ans · Mobile-first · Design immersif horror · RGAA 60%+ · RGPD

### Visiteur (non connecté)

- Parcourir toutes les activités
- Filtrer par catégorie
- Voir le détail d'une activité (description, prix, sessions)
- S'inscrire avec email + mot de passe
- Se connecter / Se déconnecter
- Navigation mobile-first responsive

### Membre (connecté)

- Tout ce que peut faire le Visiteur
- Gérer son panier (add/remove, quantité)
- Voir le total HT + TVA + TTC en temps réel
- Passer commande → statut Pending → Confirmed
- Annuler une commande en statut Pending
- Consulter l'historique de ses commandes
- Modifier son profil, changer son mot de passe
- Supprimer son compte (soft delete RGPD)

# 03 — Cahier des charges — Admin & Contraintes

## Administrateur

- ▶ CRUD Activités + upload bannière & miniature WebP
- ▶ CRUD Sessions — blocage si commandes liées
- ▶ CRUD Catégories — blocage si liée à une activité
- ▶ Lire / modifier tous les utilisateurs
- ▶ Lire / modifier le statut de toutes les commandes

## Contraintes techniques

- ▶ JWT httpOnly — jamais localStorage / sessionStorage
- ▶ Validation Zod sur tous les inputs
- ▶ argon2id pour les mots de passe
- ▶ 151 tests automatisés + CI verte obligatoire
- ▶ Docker multi-stage · GitHub Actions CI/CD

## Contraintes qualité & conformité

Responsive mobile-first

RGAA 2.1 → 60%+

RGPD (soft delete  
users/orders)

SEO (slugs auto)

Éco-conception (WebP,  
pagination, lazy loading)

# 04 — Gestion de projet SCRUM

## Planning des 5 sprints

### Sprint 1

Setup & API base

Monorepo, Prisma, migrations, API REST, seeding

### Sprint 2

Authentification

JWT httpOnly, register/login/refresh/logout, middlewares

### Sprint 3

Orders & Frontend

Commandes, panier Zustand, espace client, apiFetch 401

### Sprint 4

Admin & Deploy

Backoffice CRUD, Docker multi-stage, CI/CD, VPS Ionos

### Sprint 5

Tests & Hardening

151 tests Vitest+Supertest, audit sécurité, Swagger

## Rituels Scrum (solo)

- ▶ Product Backlog — issues GitHub labellisées
- ▶ Sprint Planning hebdomadaire
- ▶ Daily Stand-up → journal de bord
- ▶ Sprint Review + Retrospective
- ▶ DoD : lint ✓ + tests ✓ + PR mergée + CI ✓

## Traçabilité issue → commit → PR

- ▶ Branche dédiée par issue
- ▶ Commits conventionnels EN (feat/fix/chore...)
- ▶ PR mergée avec référence closes #N
- ▶ CI verte obligatoire avant merge
- ▶ Exemple réel : PR #22 — suppression deleted\_at

# 05 — Architecture logicielle multicouche

## Présentation

*React SPA*

React 19 + Vite · React Router 7 · Zustand · apiFetch intercepteur 401 · cookies httpOnly (jamais localStorage)

## Métier

*Express API*

Express 5 · TypeScript · routes → middlewares (Zod, requireAuth, requireRole) → controllers → helpers

## Accès données

*Prisma ORM*

Prisma 5 · requêtes paramétrées · select explicite · \$transaction · soft delete RGPD

## Persistence

*PostgreSQL 16*

Contraintes FK · normalisation 3NF · BDD test isolée zombiezone\_test · migrations versionnées

## Sécurité

*Transversale*

JWT httpOnly+Secure+SameSite=Strict · argon2id · Zod · CORS strict · OWASP Top 10

# 06 — Modèle de données (MCD)

## Entités et cardinalités

|            |   |              |           |                                     |
|------------|---|--------------|-----------|-------------------------------------|
| roles      | → | users        | 1,1 — 0,N | Un user a exactement un rôle        |
| users      | → | RefreshToken | 1,1 — 0,N | Multi-appareils — token par device  |
| users      | → | orders       | 1,1 — 0,N | Un user peut avoir N commandes      |
| orders     | → | orders_lines | 1,1 — 1,N | Commande = ≥1 ligne                 |
| sessions   | → | orders_lines | 1,1 — 0,N | Session référencée dans N lignes    |
| activities | → | sessions     | 1,1 — 0,N | Activité → N sessions planifiées    |
| activities | → | categories   | 0,N — 0,N | M-N via table activities_categories |

## Choix de conception notables

Soft delete (deleted\_at) sur users & orders uniquement — RGPD & audit · Hard delete sur activities/sessions/categories — libère les images · unit\_price\_ht + vat\_rate figés dans orders\_lines — historique immuable des prix · token\_id UUID dans RefreshToken — lookup O(1), comparaison argon2.verify · slug auto-généré — URLs SEO-friendly



# 07 — Stack technique justifiée

## Node.js 22 + Express 5 · Back-end

Runtime JS natif async/await, Express 5 : meilleure gestion erreurs async, middleware chain

## Prisma 5 · ORM

Schéma déclaratif, migrations versionnées, client typé, select explicite anti-fuite données

## Zod + argon2id · Sécurité

Zod : validation runtime tous inputs. argon2id : résistant GPU/ASIC — recommandé OWASP 2023

## Zustand · State

Store minimaliste sans boilerplate, hydration safe, persist optionnel (basket uniquement)

## TypeScript 5 strict · Full-stack

Typage statique, zéro any, détection erreurs à la compilation, interfaces partagées back/front

## PostgreSQL 16 · Base de données

SGBDR ACID, FK robustes, idéal pour données relationnelles à forte cohérence

## React 19 + Vite 6 · Front-end

SPA performante, HMR natif, code splitting, tree-shaking, React concurrent features

## Biome 2.x · Qualité

Linter + formateur unifié back + front, ~10x plus rapide qu'ESLint+Prettier

## Vitest + Supertest

Tests

Runner TypeScript natif, Supertest monte Express en mémoire sans port réel

# 08 — Authentification JWT — Réalisation



## Implémentation backend

- ▶ Access token 15 min — Refresh token 7 j
- ▶ Cookies httpOnly + Secure + SameSite=Strict
- ▶ Rotation systématique à chaque /refresh
- ▶ token\_id UUID stocké en BDD → révocation
- ▶ Middleware requireAuth : guard clause rapide
- ▶ requireRole : vérification rôle après auth
- ▶ Réponse anti-oracle : même msg email/mdp

## apiFetch — Frontend

- ▶ Intercepteur 401 transparent pour l'UI
- ▶ POST /auth/refresh → retry la requête initiale
- ▶ Promise singleton : refreshPromise
- ▶ Évite race condition React StrictMode
- ▶ 2 appels simultanés → même Promise partagée
- ▶ credentials: include sur toutes les requêtes
- ▶ Logout → localStorage.removeItem basket

# 09 — Réalisations — Code & Sécurité

## OWASP Top 10 — couverture

- ▶ A01 Broken Access Control → requireAuth + requireRole
- ▶ A02 Crypto Failures → argon2id + httpOnly + HTTPS
- ▶ A03 Injection → Zod validation + Prisma paramétré
- ▶ A07 Auth Failures → rotation tokens + révocation BDD
- ▶ A09 Logging → morgan + gestion erreurs typées

## Composants d'accès données (CP8)

- ▶ \$transaction Prisma : createOrder (verif. capacité atomique)
- ▶ select explicite : password\_hash jamais retourné
- ▶ getPagination helper réutilisable (offset/limit)
- ▶ Soft delete RGPD : deleted\_at sur users/orders
- ▶ Hard delete activities/sessions → libère images

## Interface React — réalisations notables

- ▶ Guard bfcache : Cache-Control no-store + vérif. réseau
- ▶ Basket Zustand : persist localStorage (non sensible)
- ▶ Upload WebP bannière + miniature via Sharp
- ▶ Pagination 12 éléments / page côté API
- ▶ Slug auto-généré : SEO-friendly sans dépendance nom

# 10 — Plan de tests — 151 tests automatisés

151

Tests  
total

116

Tests  
intégration

35

Tests  
unitaires

100%

CI verte  
→ deploy

## Tests d'intégration — Supertest

- `auth.test.ts` — 18 tests (login, refresh, logout, révocation)
- `activities.test.ts` — CRUD complet + pagination + filtres
- `orders.test.ts` — `createOrder` + vérif. capacité sessions
- `users.test.ts` — soft delete, ownership, rôles
- BDD test isolée — `resetDatabase()` avant chaque suite
- `request.agent(app)` pour cookies persistants

## Tests unitaires — Vitest

- `tokens.test.ts` — `generateAccessToken` / `generateRefreshToken`
- `auth.test.ts` — `hashPassword` / `verifyPassword` (argon2id)
- `slugify.test.ts` — cas nominaux + accents + collisions
- `getPagination.test.ts` — `offset/limit/page/perPage`
- Couverture : cas nominaux + limites + erreurs attendues
- Scripts : `test:unit`, `test:integration`, `test:coverage`

# 11 — Jeu d'essai — Cycle JWT complet

**S1**

Login valide

✓ 200 OK — 2 cookies httpOnly définis (access + refresh)

**S2**

Mauvais mot de passe

✓ 401 — 'Invalid credentials' (même msg email inconnu → anti-oracle)

**S3**

Token expiré → Refresh auto

✓ 401 → POST /refresh → rotation token\_id BDD → retry 200 OK

**S4**

Accès sans authentification

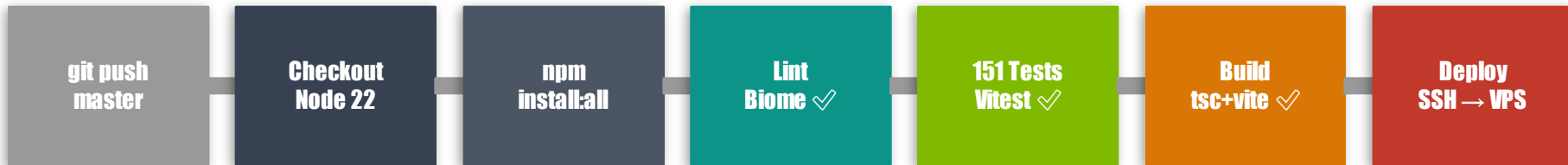
✓ 401 — 'No token' — guard clause middleware déclenché

**S5**

Logout + refreshToken révoqué

✓ Logout 200 → POST /refresh avec ancien token → 401 token\_id absent BDD

# 12 — Pipeline CI/CD — GitHub Actions



*if: refs/heads/master*

## Services CI (ephemeral)

- ▶ PostgreSQL 16 éphémère dans le runner GitHub
- ▶ zombiezone\_test — restaurée à chaque run
- ▶ Prisma migrate deploy avant les tests
- ▶ Test env séparé de dev et de prod
- ▶ Timeout : 10 min max par job

## Déclenchement & stratégie

- ▶ Déclenché sur push master + toutes les PR
- ▶ Deploy SSH uniquement sur push master
- ▶ Fail-fast : lint échoue → tests ne tournent pas
- ▶ Secrets GitHub : SSH\_KEY, DB\_URL, etc.
- ▶ Procédure de mise à jour documentée DEPLOY.md

# 13 — Déploiement — Docker & VPS

## Infrastructure VPS Ionos

- ▶ Ubuntu 24.04 LTS — 2 vCPU / 2 GB RAM / 80 GB NVMe
- ▶ Nginx : SSL termination + redirect HTTP→HTTPS
- ▶ Proxy /api/ → backend:3000 (Docker interne)
- ▶ SPA fallback : try\_files → index.html
- ▶ Let's Encrypt — renouvellement automatique Certbot
- ▶ PostgreSQL port 5432 interne Docker — non exposé
- ▶ Domaine : sharo.fr → DNS A 82.165.180.54

## Docker multi-stage

- ▶ Stage builder : node:22 + tsc → dist/
- ▶ Stage runner : node:22-alpine (dist/ uniquement)
- ▶ Image prod allégée — sans sources TypeScript
- ▶ Symlink prisma/migrations (piège résolu)
- ▶ 3 services Docker Compose prod :
  - ▶ db → PostgreSQL 16 Alpine (volume pgdata)
  - ▶ backend → Node 22 Alpine (dist/, .env.production)
  - ▶ frontend → Nginx Alpine (build Vite statique)

# 14 — Difficultés techniques résolues

## Race condition — Refresh token (React StrictMode)

⚠ Double-invoke des effets → 2 appels /auth/refresh simultanés → token déjà rotaté → 401 au 2e appel

✓ Promise singleton (let refreshPromise: Promise<void> | null = null) partagé dans apiFetch + store Zustand

## bfcache — Page protégée visible après déconnexion

⚠ Bouton 'Précédent' restaure la page depuis la mémoire du navigateur — aucun JS ne s'exécute

✓ Cache-Control: no-store sur les routes protégées + guard réseau au montage de chaque page d'espace client

## Docker multi-stage — Migrations Prisma introuvables

⚠ prisma migrate deploy cherche prisma/migrations relatif au CWD (/app/backend) — 'No migration found' en prod

✓ Symlink backend/prisma/migrations → backend/src/models/migrations dans le Dockerfile — documenté DEPLOY.md

## Biome 2.x — Conflit config en monorepo

⚠ Exécution depuis la racine → conflit biome.json racine vs vite-frontend/biome.json → erreur CI

✓ Exécution depuis chaque workspace (cd vite-frontend && npx biome check src/) — documenté CLAUDE.md



# 15 — Améliorations futures & axes V2

## Rate limiting

Haute

express-rate-limit sur `/api/auth/*` — protection anti-brute-force. Identifié lors du jeu d'essai.

## Emails transactionnels

Haute

Nodemailer / Resend : confirmation inscription + commande (lignes HT + total TTC).

## Paiement Stripe

Moyenne

Intégration Stripe pour confirmation de commande — prévu V2 (hors MVP selon cahier des charges).

## Tests E2E Playwright

Moyenne

Flow complet navigateur : login → réservation → confirmation → annulation.

## i18n FR/EN

Basse

Support multilingue prévu dans l'architecture (routes `/fr` et `/en`), non implémenté.

## RGAA complet

Basse

Audit accessibilité pour dépasser 60% — tests lecteur d'écran, navigation clavier complète.

# 16-17 — Compétences CDA — 11 CP couvertes

## AT1 — Développer une appli sécurisée

|            |                                                 |
|------------|-------------------------------------------------|
| <b>CP1</b> | Env. & config<br>Monorepo, Docker, Biome, CI/CD |
|------------|-------------------------------------------------|

|            |                                                |
|------------|------------------------------------------------|
| <b>CP2</b> | UI<br>React 19, TypeScript, mobile-first, RGAA |
|------------|------------------------------------------------|

|            |                                                  |
|------------|--------------------------------------------------|
| <b>CP3</b> | Composants métier<br>Express 5, Zod, middlewares |
|------------|--------------------------------------------------|

|            |                                                      |
|------------|------------------------------------------------------|
| <b>CP4</b> | Gestion projet<br>SCRUM, GitHub Projects, commits EN |
|------------|------------------------------------------------------|

## AT2 — Concevoir & développer (couches)

|            |                                                                 |
|------------|-----------------------------------------------------------------|
| <b>CP5</b> | Analyser & maquetter<br>Cahier des charges, user stories, Figma |
|------------|-----------------------------------------------------------------|

|            |                                                      |
|------------|------------------------------------------------------|
| <b>CP6</b> | Architecture<br>Multicouche SPA↔API↔BDD, traçabilité |
|------------|------------------------------------------------------|

|            |                                                           |
|------------|-----------------------------------------------------------|
| <b>CP7</b> | Base de données<br>PostgreSQL, MCD/MPD, Prisma migrations |
|------------|-----------------------------------------------------------|

|            |                                                         |
|------------|---------------------------------------------------------|
| <b>CP8</b> | Accès données<br>Prisma ORM, \$transaction, soft delete |
|------------|---------------------------------------------------------|

## AT3 — Préparer le déploiement

|            |                                               |
|------------|-----------------------------------------------|
| <b>CP9</b> | Plan de tests<br>151 tests Vitest + Supertest |
|------------|-----------------------------------------------|

|             |                                                           |
|-------------|-----------------------------------------------------------|
| <b>CP10</b> | Déploiement & doc<br>DEPLOY.md, Docker, Swagger, TESTS.md |
|-------------|-----------------------------------------------------------|

|             |                                                 |
|-------------|-------------------------------------------------|
| <b>CP11</b> | DevOps<br>GitHub Actions, Nginx, SSL, VPS Ionos |
|-------------|-------------------------------------------------|

# z0mbie z0ne

*Application fullstack TypeScript — déployée en production*

11

Compétences  
CDA couvertes

151

Tests  
automatisés

26

Endpoints  
API REST

79,99%

CI/CD  
automatisé

---

*Stack : Express 5 · Prisma · PostgreSQL · React 19 · Vite · Zustand · Docker · GitHub Actions*

🔗 Production : [sharo.fr](https://sharo.fr)

📖 API docs : [sharo.fr/api/docs](https://sharo.fr/api/docs)

💻 Code : [github.com/5h4r0/z0mbiez0ne](https://github.com/5h4r0/z0mbiez0ne)

**Merci — Questions ?**